Assignment No: 5

Name: Atharv Shinde
Class: T.Y IT-A
Batch: D
Roll No: 33076
Subject: DSML

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix

# 1. Load and Prepare the Dataset
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

# Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# 2. Feature Scaling
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. Model: SVM with a Linear Kernel
linear_svm = SVC(kernel='linear', C=1.0, random_state=42)
linear_svm.fit(X_train_scaled, y_train)
y_pred_linear = linear_svm.predict(X_test_scaled)

# 4. Performance Evaluation & Analysis
def evaluate_model(model_name, y_true, y_pred):
    accuracy = accuracy_score(y_true, y_pred)
    print(f"================ {model_name} Performance
================")
    print(f"Accuracy: {accuracy:.4f}\n")
    print("Classification Report:")
    print(classification_report(y_true, y_pred, target_names=target_names))

    # Plot Confusion Matrix
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=target_names, yticklabels=target_names)
    plt.title(f'Confusion Matrix: {model_name}')
    plt.xlabel('Predicted Label')
    plt.ylabel('True Label')
    plt.show()

# Run evaluation
evaluate_model("Linear SVM", y_test, y_pred_linear)
```

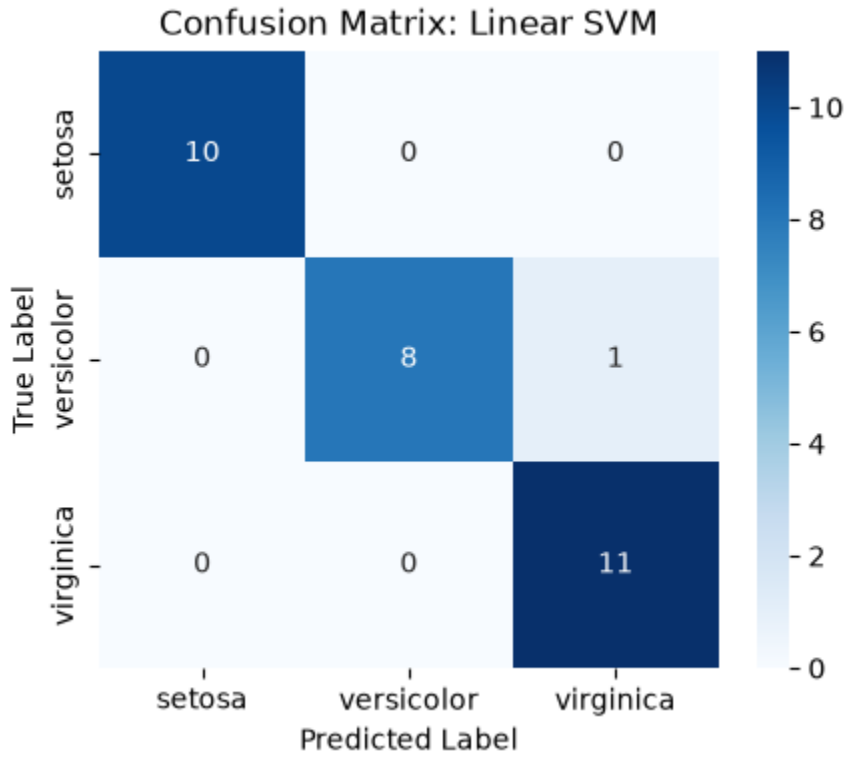
O/p:

```
================ Linear SVM Performance ================
Accuracy: 0.9667

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.89      0.94         9
```

virginica	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

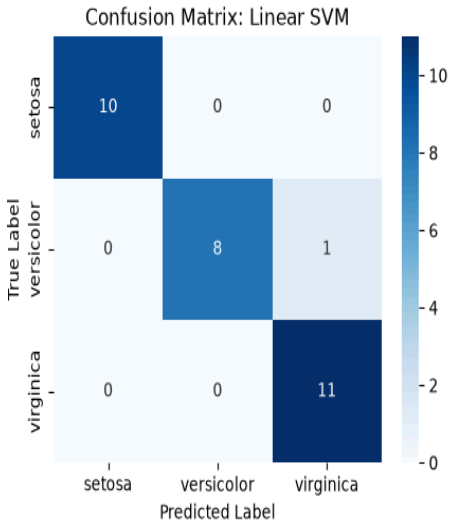


```
================ Linear SVM Performance ================
Accuracy: 0.9667

Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.89      0.94         9
   virginica       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```



AFTER CODE FOR SVM+DECISON TREE and COMPARE MODEL

Code:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    precision_recall_fscore_support,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

# ========================================
# 1. Load and Prepare the Dataset
# ========================================
iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

# Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)

# Shared evaluation plotting helper
def plot_cm(cm, model_name):
    plt.figure(figsize=(5, 4))
    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Blues",
        xticklabels=target_names,
        yticklabels=target_names,
    )
    plt.title(f"Confusion Matrix: {model_name}")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.show()


# ========================================
# 2. Linear SVM Model
# ========================================
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

svm_model = SVC(kernel="linear", C=1.0, random_state=42)
svm_model.fit(X_train_scaled, y_train)
y_pred_svm = svm_model.predict(X_test_scaled)

print("================ Linear SVM Performance ================")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}\n")
print(classification_report(y_test, y_pred_svm, target_names=target_names))
plot_cm(confusion_matrix(y_test, y_pred_svm), "Linear SVM")


# ========================================
# 3. Decision Tree Classifier Model
# ========================================
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)  # Scale-invariant
y_pred_dt = dt_model.predict(X_test)

print("================ Decision Tree Performance ================")
print(f"Accuracy: {accuracy_score(y_test, y_pred_dt):.4f}\n")
print(classification_report(y_test, y_pred_dt, target_names=target_names))
plot_cm(confusion_matrix(y_test, y_pred_dt), "Decision Tree")


# ========================================
# 4. Comparative Metrics Analysis
# ========================================
# Compute accuracy and misclassifications
acc_svm = accuracy_score(y_test, y_pred_svm)
acc_dt = accuracy_score(y_test, y_pred_dt)

errors_svm = np.sum(y_test != y_pred_svm)
errors_dt = np.sum(y_test != y_pred_dt)
```

```
# Compute macro precision, recall, and F1-scores with consistent variable
names
_, _, f1_svm, _ = precision_recall_fscore_support(
    y_test, y_pred_svm, average="macro"
)
_, _, f1_dt, _ = precision_recall_fscore_support(
    y_test, y_pred_dt, average="macro"
)

print("=== METRIC-BY-METRIC COMPARISON ===\n")

# CONDITION 1: Accuracy Comparison
if acc_dt > acc_svm:
    print(f"1. Accuracy: Decision Tree is better ({acc_dt:.4f} vs {acc_svm:.4f})")
elif acc_svm > acc_dt:
    print(f"1. Accuracy: Linear SVM is better ({acc_svm:.4f} vs {acc_dt:.4f})")
else:
    print(f"1. Accuracy: Both models perform equally ({acc_svm:.4f})")

# CONDITION 2: Misclassifications Comparison
if errors_dt < errors_svm:
    print(
        f"2. Error Count: Decision Tree is better with fewer errors ({errors_dt} vs
{errors_svm})"
    )
elif errors_svm < errors_dt:
    print(
        f"2. Error Count: Linear SVM is better with fewer errors ({errors_svm} vs
{errors_dt})"
    )
else:
    print(f"2. Error Count: Both models made equal errors ({errors_svm})")

# CONDITION 3: Macro F1-Score Comparison
if f1_dt > f1_svm:
    print(
        f"3. Macro F1-Score: Decision Tree is better ({f1_dt:.4f} vs {f1_svm:.4f})"
```

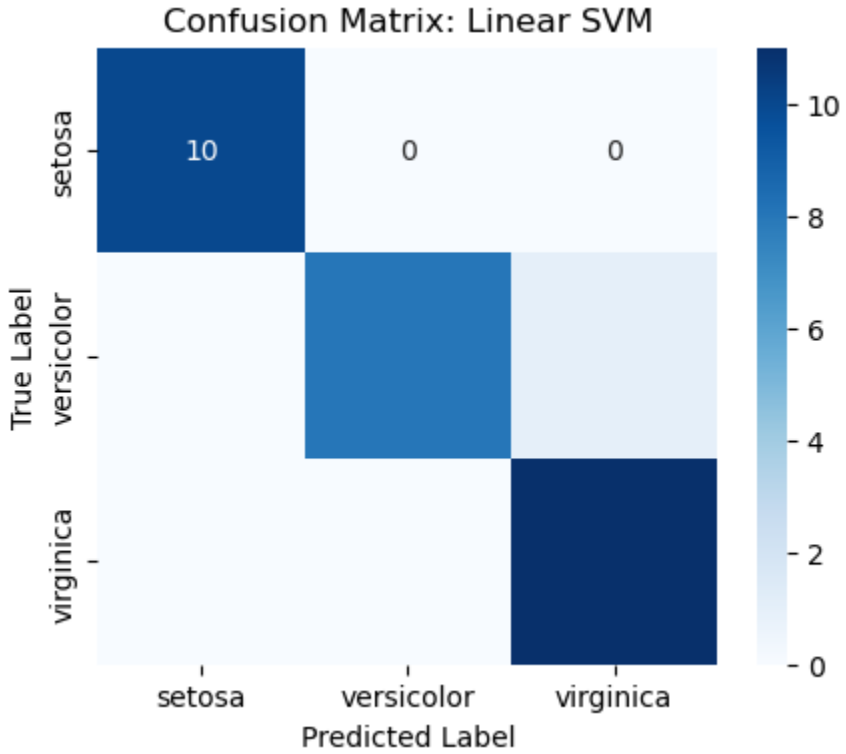
```
    )
elif f1_svm > f1_dt:
    print(f"3. Macro F1-Score: Linear SVM is better ({f1_svm:.4f} vs {f1_dt:.4f})")
else:
    print(f"3. Macro F1-Score: Both models perform equally ({f1_svm:.4f})")
```

O/p:

========== Linear SVM Performance ==========

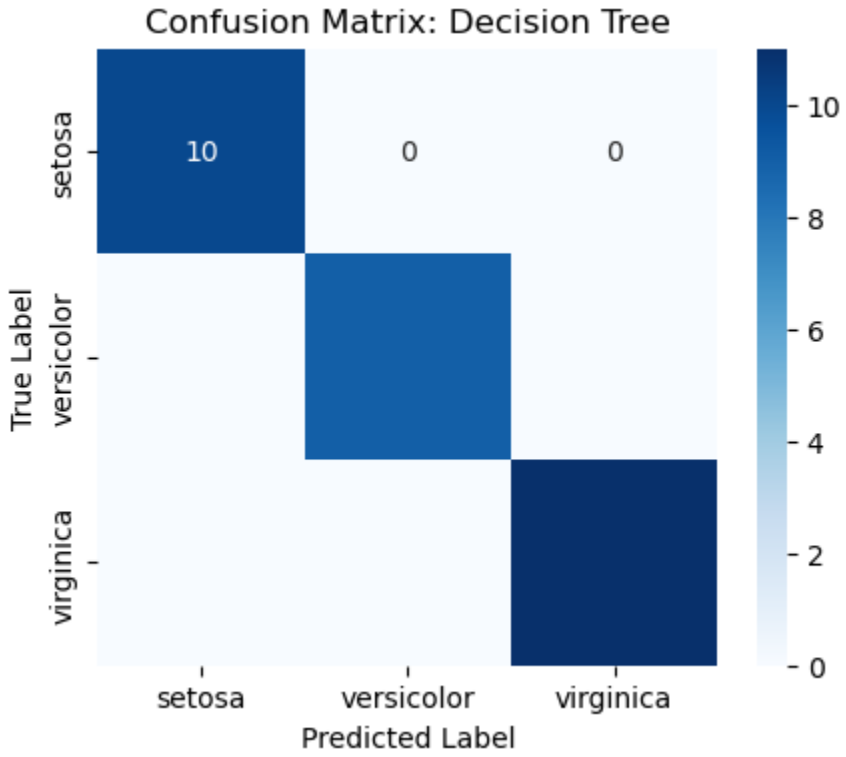
Accuracy: 0.9667

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	0.89	0.94	9
virginica	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30



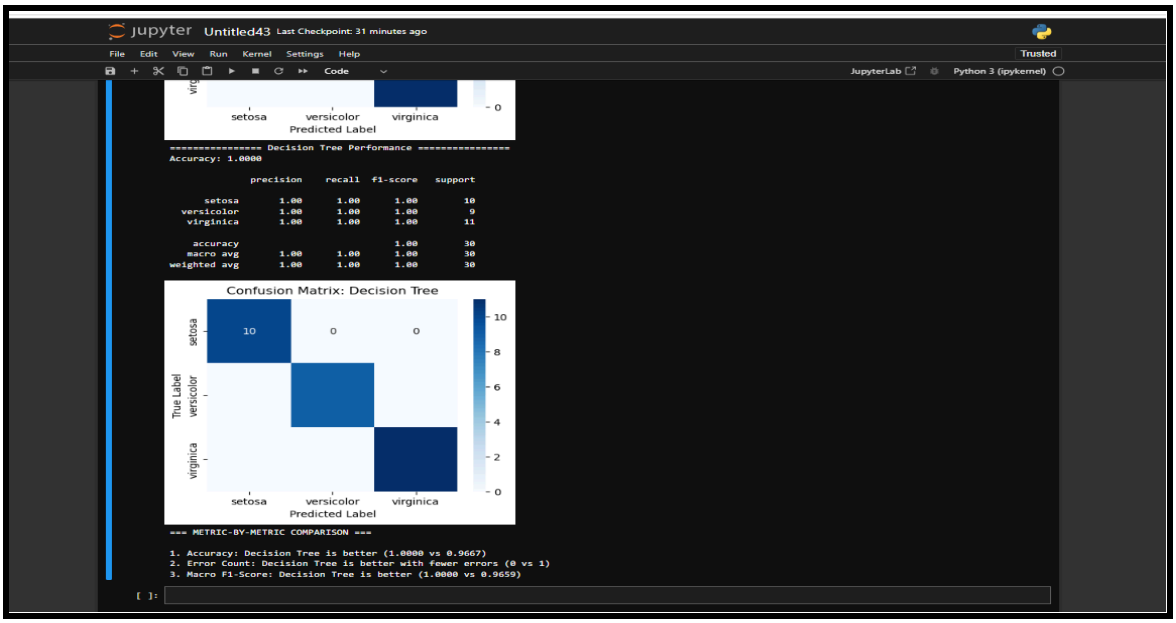
========== Decision Tree Performance ==========
Accuracy: 1.0000

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



=== METRIC-BY-METRIC COMPARISON ===

1. Accuracy: Decision Tree is better (1.0000 vs 0.9667)
2. Error Count: Decision Tree is better with fewer errors (0 vs 1)
3. Macro F1-Score: Decision Tree is better (1.0000 vs 0.9659)


jupyter
Untitled43
Last Checkpoint: 31 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
setosa
versicolor
virginica
Predicted Label
0
================ Decision Tree Performance ================
Accuracy: 1.0000
precision recall f1-score support
setosa 1.00 1.00 1.00 10
versicolor 1.00 1.00 1.00 9
virginica 1.00 1.00 1.00 11
accuracy 1.00 30
macro avg 1.00 1.00 1.00 30
weighted avg 1.00 1.00 1.00 30
Confusion Matrix: Decision Tree
True Label
10
0
0
10
8
6
4
2
0
=== METRIC-BY-METRIC COMPARISON ===
1. Accuracy: Decision Tree is better (1.0000 vs 0.9667)
2. Error Count: Decision Tree is better with fewer errors (0 vs 1)
3. Macro F1-Score: Decision Tree is better (1.0000 vs 0.9659)
[ ]: